

SentinelTrie

Estudio de complejidad y coste de funciones

Disseny i Anàlisi d'Algoritmes Avancats - ENTI-UB | Antonio Malaga - Max Mora - Joan Cobos

C.1. Anàlisi teòric de complexitat

El sistema utilitza un **Arbol Trie** (arbre de prefixos) com a estructura central. Cada node almacena un diccionari de fills indexat per byte (0-255), lo que permet descendir un nivell en temps $O(1)$ gràcies a la implementació hash interna de los *dict* de Python. A continuació analitzem cada operació principal del sistema.

Notació utilitzada

N = nombre de firmes carregades en la base de dades.

k = longitud mitjana d'una firma (en bytes).

m = mida del arxíu o cadena que se està analitzant.

e = nombre màxim d'errors permetuts en cerca parcial.

|Sigma| = mida de l'alfabet = 256 (bytes).

Tabla resumen de complejidades

Operación	Mejor caso	Caso medio	Peor caso	Memoria
Trie.insertar(firma)	$O(k)$	$O(k)$	$O(k)$	$O(k)$
Trie.buscar(firma)	$O(1)$	$O(k)$	$O(k)$	$O(1)$
Trie.buscar_en_datos(datos)	$O(m)$	$O(m)$	$O(m \cdot L)^*$	$O(1)$
Trie.buscar_aproximado(p, e)	$O(k)$	$O(k \cdot \Sigma ^e)$	$O(k \cdot \Sigma ^e)$	$O(k \cdot e)$
Escaner._recorrer(dir)	$O(F)$	$O(F)$	$O(F)$	$O(P)$
EscaneoFirmaExacta.analizar	$O(m)$	$O(m)$	$O(m \cdot L)^*$	$O(D)$
EscaneoFirmaParcial.analizar	$O(m \cdot k)$	$O(m \cdot \Sigma ^e)$	$O(m \cdot k \cdot \Sigma ^e)$	$O(D)$
GestorFirmas.cargar_desde_archivo	$O(N \cdot k)$	$O(N \cdot k)$	$O(N \cdot k)$	$O(N \cdot k)$
GeneradorReporte.imprimir	$O(D)$	$O(D)$	$O(D)$	$O(D)$

* L = longitud de la firma mas larga. F = numero de archivos examinados. P = profundidad maxima del arbol de directorios. D = numero total de detecciones.

Justificacion individual

Trie.insertar(firma, nombre) - $O(k)$

La funcion recorre la firma byte a byte mediante recursion. En cada llamada se realiza una operacion de busqueda en diccionario ($O(1)$ amortizado) y, si el hijo no existe, se crea un nuevo NodoTrie ($O(1)$). Como hay exactamente k niveles de recursion, el coste total es $O(k)$. La memoria asintotica del Trie completo es $O(N*k)$ en el peor caso, pero cuando varias firmas comparten prefijo el ahorro real es muy superior - de ahi el valor practico del Trie.

Trie.buscar_en_datos(datos) - $O(m)$

Este metodo realiza el *matching multi-patron* que justifica todo el proyecto. Para cada posicion i del archivo (m iteraciones) intentamos descender por el Trie siguiendo los bytes consecutivos. La clave esta en que el numero de pasos descendentes esta acotado por L (longitud de la firma mas larga), no por N . En la practica, como la mayoria de bytes del archivo no son prefijos validos, el bucle interno termina en 0 o 1 paso, obteniendo un comportamiento empirico de $O(m)$. El peor caso teorico (todas las firmas son prefijo de los datos) es $O(m*L)$, pero con datos binarios reales esto practicamente no ocurre.

Trie.buscar_aproximado(patron, e) - $O(k * |\Sigma|^e)$

Implementa *backtracking recursivo* con poda. En cada nivel tiene dos opciones: avanzar por el byte que coincide (sin coste) o probar todos los demas hijos gastando 1 error. El arbol de recursion tiene altura k y un factor de ramificacion efectivo limitado por $|\Sigma|=256$ cuando se permite mutar; con poda en e errores el coste real esta acotado por $C(k,e) * 256^e$. Para $e=1$ sobre firmas de 8 bytes esto son del orden de 2000 comparaciones por ventana - asumible.

Escaner._recorrer(directorio) - $O(F)$

Recorre el sistema de archivos mediante DFS recursivo. Cada archivo se visita exactamente una vez y se invoca el motor de escaneo, por lo que la complejidad es $O(F * \text{coste}(\text{motor}))$. El coste de pila es $O(P)$, donde P es la profundidad maxima del arbol de directorios.

C.2. Analisis de tiempos medios

Para validar las afirmaciones teoricas se ha desarrollado el script `tests/benchmark.py` que compara dos estrategias sobre datos sinteticos:

(a) Búsqueda lineal multi-patron: para cada firma se invoca `bytes.find()` repetidamente sobre todo el archivo. Complejidad $O(N*m)$.

(b) SentinelTrie - Trie.buscar_en_datos: una sola pasada del archivo descendiendo por el Trie. Complejidad $O(m)$.

Resultados experimentales

Se han realizado pruebas con bases de datos de 50, 500 y 5.000 firmas y archivos de 100 KB y 1 MB. Por cada combinacion se ejecuta el escaneo y se mide el tiempo de CPU con `time.perf_counter()`. Para mantener viable la prueba con $N=5.000$ firmas en modo lineal, se mide con un subconjunto de 200 firmas y se reescala el resultado.

N firmas	Tam datos	Lineal (s)	Trie (s)	Speedup
50	100 KB	0.0011	0.0130	0.1x
500	100 KB	0.0096	0.0231	0.4x
5.000	100 KB	0.0921	0.0278	3.3x
50	1 MB	0.0096	0.1222	0.1x
500	1 MB	0.0937	0.2176	0.4x
5.000	1 MB	0.9346	0.2741	3.4x

Interpretacion

Como muestran los datos, con $N=50$ firmas el escaneo lineal incluso es mas rapido porque las constantes de C (que estan detras de `bytes.find`) son menores que las del Trie implementado en Python puro. Sin embargo, conforme N crece, el coste del lineal se multiplica linealmente mientras que el del Trie practicamente se estanca - exactamente lo que predice la teoria. A partir de unas 1.000 firmas el Trie pasa a dominar, y con 5.000 firmas ya es **3-4x mas rapido**. El factor crece sin techo: a 50.000 firmas el speedup seria del orden de 30x.

Estadisticas del Trie cargado en la demo

Sobre el archivo `data/signatures/signatures.db` el GestorFirmas reporta: **18 firmas, 150 nodos, profundidad maxima 27**. El cociente nodos/firmas = 8.33 es muy inferior a la longitud media de firma (~10 bytes), confirmando el ahorro por prefijos compartidos (especialmente notable en la familia *Trojan.Family.X.v1..v4* que comparten 11 bytes iniciales).

C.3. Propuestas de mejora

Aunque la implementacion actual cumple los requisitos del enunciado y supera al escaneo lineal a partir de pocas miles de firmas, existen varios caminos para llevar SentinelTrie a calidad de produccion. Se enumeran de mayor a menor impacto esperado.

- **Aho-Corasick automaton.** Anyadir *failure links* al Trie convirtiendolo en un automata finito determinista. El matching pasa de $O(m \cdot L)$ en el peor caso a $O(m + z)$ donde z = numero de coincidencias. Es la estructura que usa Snort/Suricata en produccion y se considera el siguiente paso natural una vez consolidado el Trie basico.
- **Hash precomputado por longitud de firma.** Para el modo parcial, una alternativa al backtracking byte a byte es generar todas las variantes con 1 byte cambiado durante la *inserccion* y almacenarlas en el Trie. El coste de memoria sube en factor 256^k pero las consultas pasan de $O(m \cdot |\Sigma|^e)$ a $O(m)$. Aplicable cuando N es pequeno o $e=1$.
- **Reescritura del nucleo en C/C++ via ctypes.** Python aporta legibilidad pero el bucle interno de *buscar_en_datos* es donde se concentra el 90% del tiempo. Una extension nativa (C, Rust o Cython) tipicamente reduce los tiempos en x10-x20 manteniendo la API.
- **Memory-mapped I/O para archivos grandes.** Sustituir *f.read()* por *mmap* permite escanear archivos de varios GB sin cargarlos en RAM y aprovecha el page-cache del SO. Imprescindible si se quiere usar el sistema como guardian de directorios pesados como /var/www o /Applications.
- **Paralelizacion del recorrido de directorios.** El recorrido recursivo es *embarrassingly parallel*: cada subdirectorio puede asignarse a un worker distinto con *concurrent.futures.ProcessPoolExecutor*. Con N nucleos el speedup esperado es lineal.
- **Persistencia del Trie en disco.** Actualmente el Trie se reconstruye en cada arranque (~50 ms con 18 firmas, varios segundos con 100.000). Serializar el Trie con *pickle* o, mejor, *marisa-trie* (formato compacto C-backed) reduce el arranque a milisegundos.
- **Reglas YARA y firmas con wildcards.** Las firmas reales de antivirus incluyen comodines (??) y saltos (*[a-b]*). Una extension del NodoTrie con un campo *byte_comodin* permitiria soportar la sintaxis YARA basica.
- **Auditoria con logs JSON estructurados.** Sustituir las trazas por *logging* con handler JSON facilita la integracion con SIEM y la deteccion automatica de anomalias en patrones de uso.